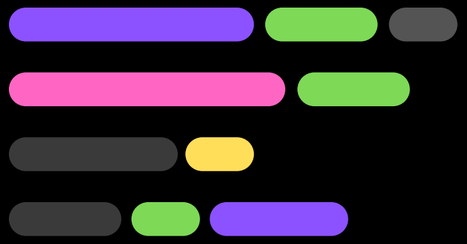
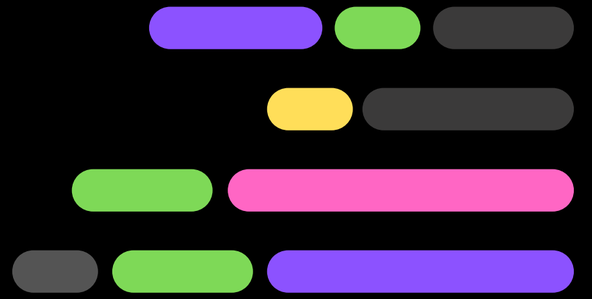
Exercise 3:

Side Channels

Information Security Lab





Lejla

What are side channels?

- Sensitive information can leak through unintended, observable effects of a computation
- An attacker exploits these observations to infer information they cannot access directly
- Side-channel attacks do not necessarily require exploiting a software vulnerability

The sound of Insecurity

Q1



Scenario

You are trying to open your safe 💰 protected by a combination lock with 4 dials. Each dial features digits from 0 to 9.

You forgot the lock combination, 😧 but fortunately, the mechanical lock imposes no limit on the number of attempts.



What is the maximum number of attempts needed to open the safe?



What is the maximum number of attempts needed to open the safe?

```
# dials                          ??
# possible values for a digit    ??
```



What is the maximum number of attempts needed to open the safe?

```
# dials                          4
# possible values for a digit   10

→ ?? possible combinations
```



What is the maximum number of attempts needed to open the safe?




```
# dials                          4
# possible values for a digit   10
```

→ 10^4 possible combinations

→ ?? attempts needed in the worst case

What is the maximum number of attempts needed to open the safe?




```
# dials                          4
# possible values for a digit   10
```

→ 10^4 possible combinations

→ 10^4 attempts needed in the worst case

What is the maximum number of attempts needed to open the safe?




```
# dials                          4
# possible values for a digit   10
```

→ 10^4 possible combinations

→ 10^4 attempts needed in the worst case
  → 10^4-1 to determine the combination

The sound of Insecurity

Q2



Scenario

You discover something interesting about the lock… You turn the first dial. When you reach digit 8, you hear a distinctive clicking sound.👂 No other digit produces the same sound.

Curious, you turn the second dial and notice something similar – it clicks when you reach 5. You repeat this process for the remaining dials, and after the 4th click, the safe opens!! 😱



What information does the clicking sound reveal?



What information does the clicking sound reveal?

It reveals whether the currently tested digit is correct.



How would you use what you learned to open another safe[1] with the same lock model?



1 that belongs to you

How would you use what you learned to open another safe with the same lock model?

Instead of trying all possible combinations, the attacker can determine each digit individually by listening for the clicking sound.



What is the maximum number of attempts needed to open the safe?

```
# dials                          4
# possible values for a digit   10

→ ?? attempts
```



What is the maximum number of attempts needed to open the safe?

```
# dials                          4
# possible values for a digit   10

→ 4 x 10 = 40 attempts
```



What is the maximum number of attempts needed to open the safe?



```
# dials                          4
# possible values for a digit   10
```

→ ~~4 x 10 = 40 attempts~~

→ 4 x 9 = 36 tests to determine the combination
  → +1 attempt to open the safe

How could the manufacturer improve the lock?



How could the manufacturer improve the lock?

→ Make the clicking sounds indistinguishable

→ Limit the number of attempts

→ Require the full combination before performing any verification



Suppose the manufacturer adds random clicking sounds as the dials are turning. Does this solve the problem?



Suppose the manufacturer adds random clicking sounds as the dials are turning. Does this solve the problem?



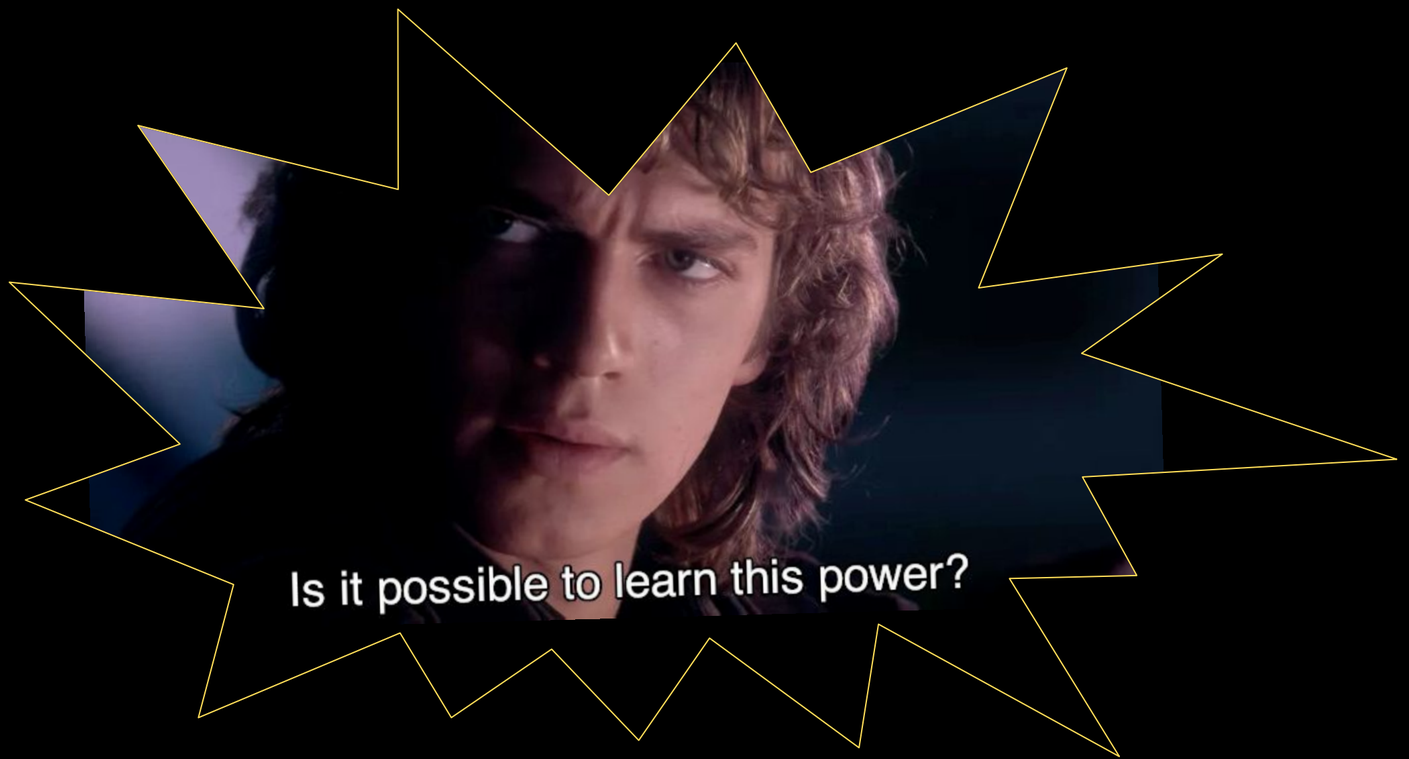
→ Not necessarily. If the correct digit still produces a statistically distinguishable sound, the attacker may be able to repeat measurements and average out the noise.

The sound of Insecurity

is an acoustic side channel

The power of Secrets

Q1



Scenario

When you were little, you had a toy instrument that could record and replay melodies. You once composed a special melody for your mom, and she loved it so much that she asked you to play it over and over again.

Her birthday is coming up, and you want to surprise her with a photo slideshow playing your melody in the background.



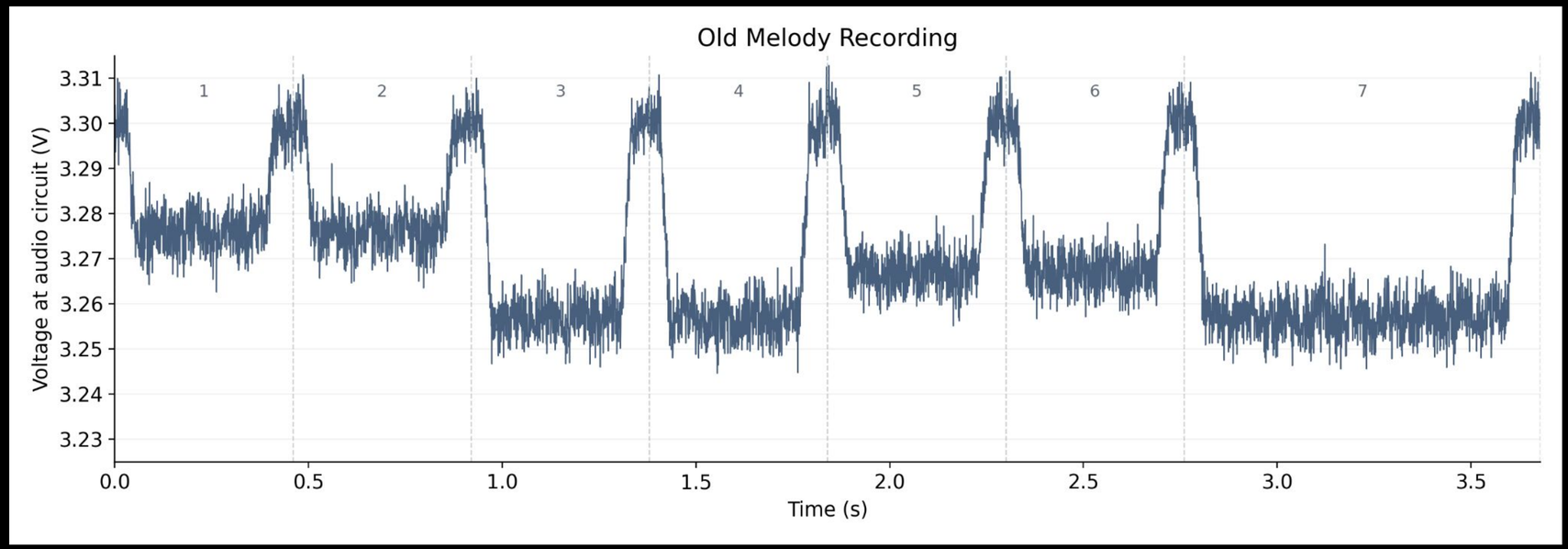
Scenario

You find the toy in the attic, but the speaker is broken!🥺 You bring the toy to the lab, connect it to an oscilloscope, and press PLAY.

The toy remains silent, but the voltage trace looks surprisingly interesting… 👀



What information might the measured voltage provide?



What information might the measured voltage provide?

→ The voltage trace contains distinct intervals with different voltage levels. These could correspond to different notes being played. Furthermore, the duration of each interval might reveal how long a note was held.



How would you confirm this?



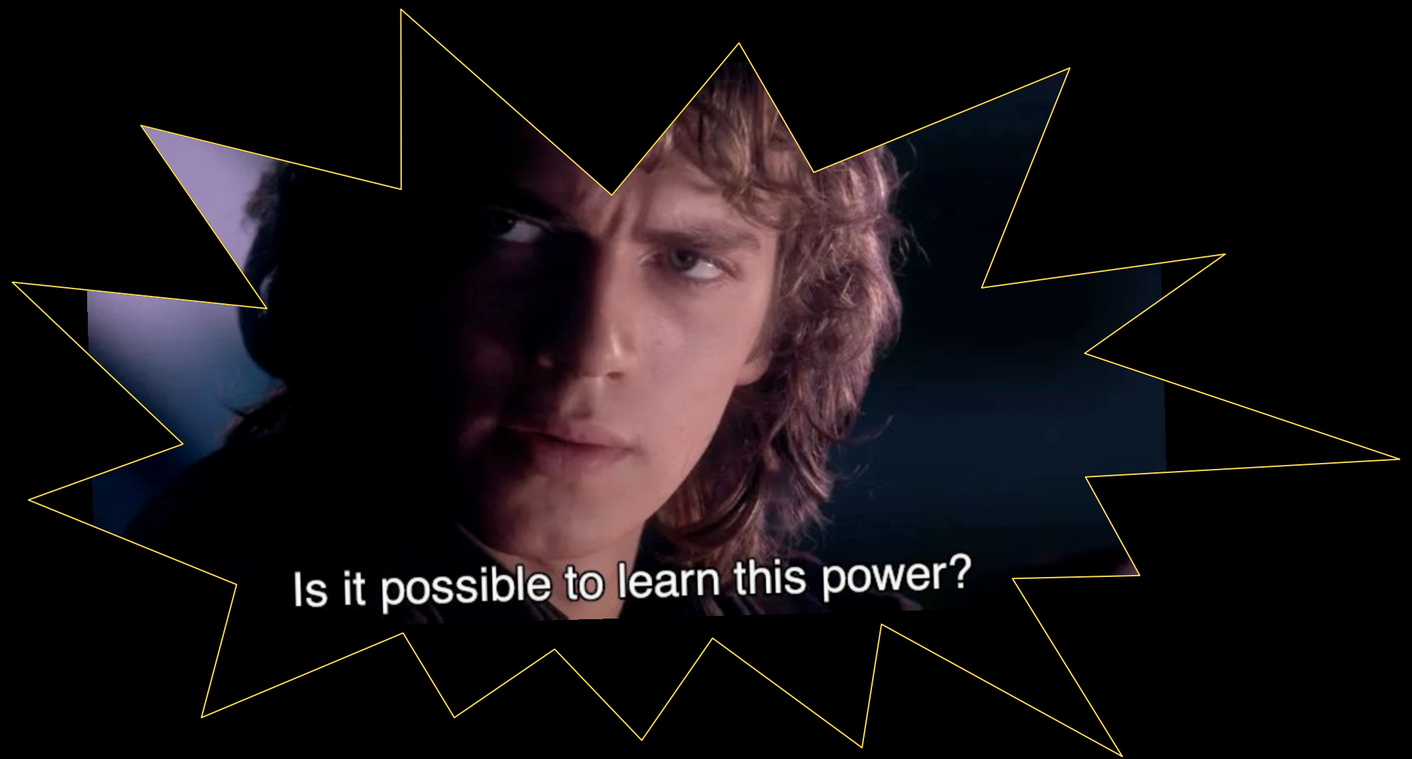
How would you confirm this?

→ Perform controlled experiments by playing individual known notes and recording their voltage traces.



The power of Secrets

Q2



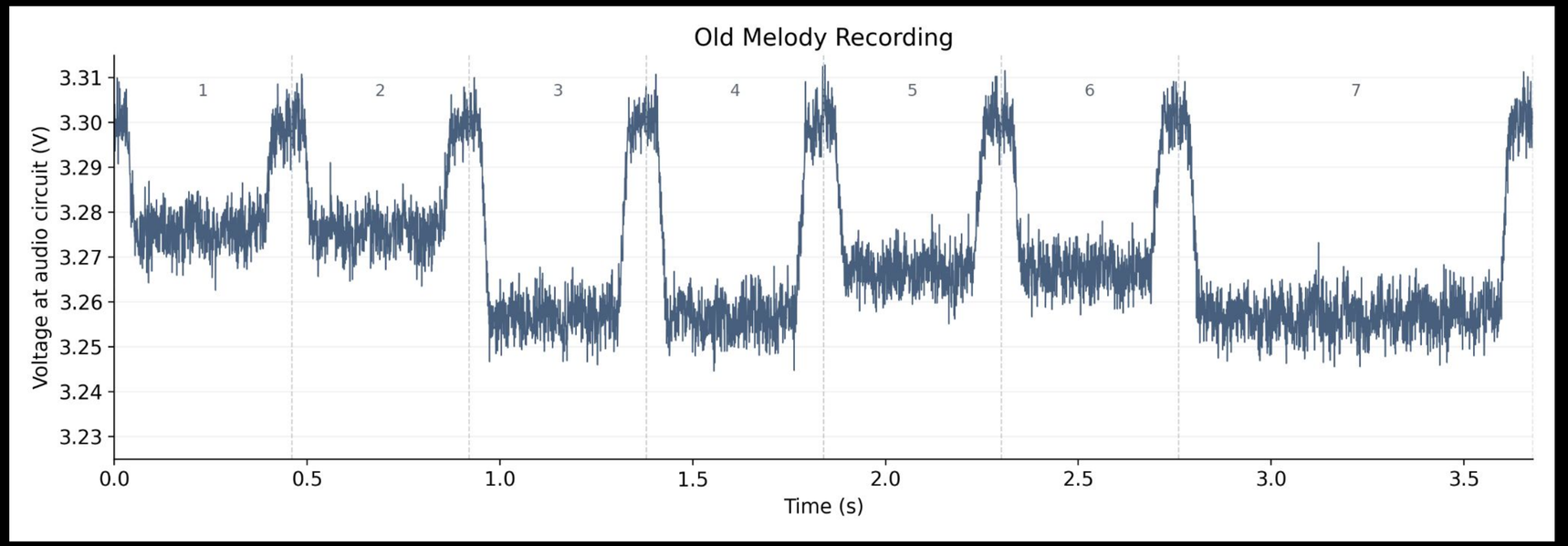
Scenario

Suppose you do that and find the following measured average voltages:

C	3.275
D	3.240
E	3.295
F	3.300
G	3.255
A	3.265
B	3.235

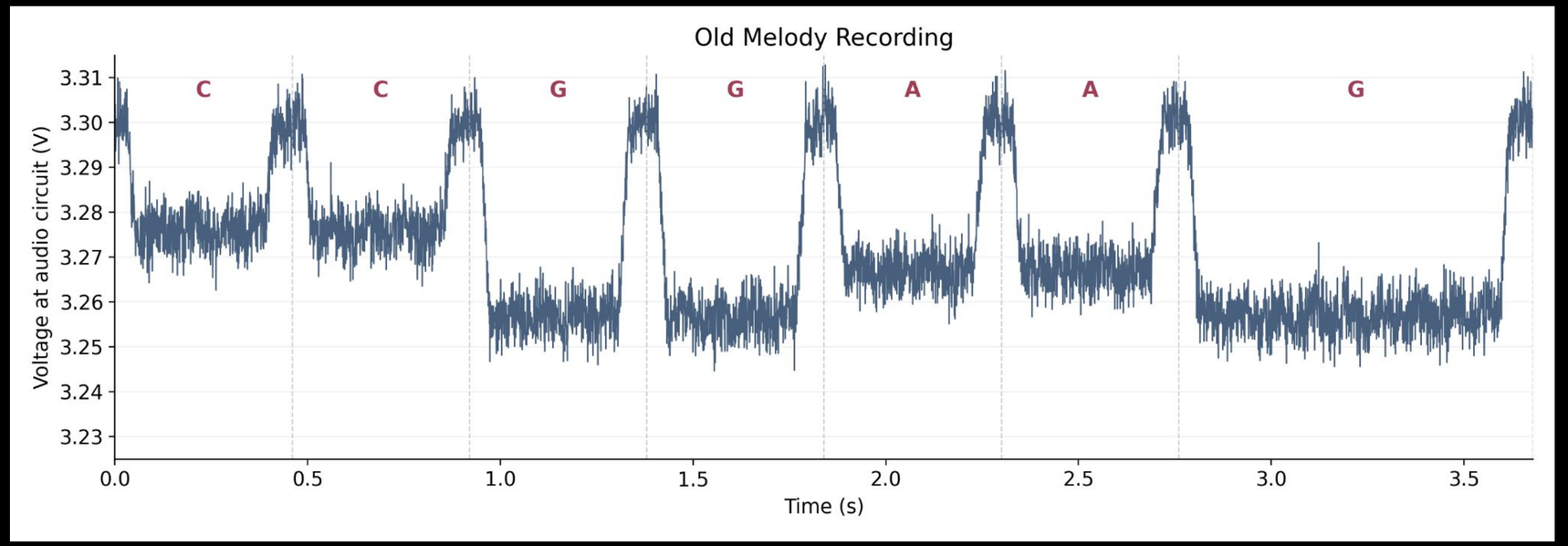


Can you recover the melody?



C	3.275
D	3.240
E	3.295
F	3.300
G	3.255
A	3.265
B	3.235

Can you recover the melody?



C	3.275
D	3.240
E	3.295
F	3.300
G	3.255
A	3.265
B	3.235

How could a similar side channel be used to extract information from a TEE?

How could a similar side channel be used to extract information from a TEE?

→ An attacker could monitor power consumption while the enclave processes secret data.

The power of Secrets

is a power side channel

The Prime Suspect

Q1



Scenario

You're running a number-theory experiment for your semester project. After several hours, your program finally finds a mysterious number! 🎉

Unfortunately, the terminal crashes before displaying it. 😭

Luckily, the program's execution-time log survived.



Scenario

Your program performs some preliminary computations which take 100 ms for each number n.

After that, the program checks whether n is prime by testing divisors in ascending order, starting at 2. It stops at the first divisor of n, or after checking all integers up to the square root of n. Each divisibility test takes exactly 2 ms.

The log reveals that the program's final computation took 108 ms in total.



Which of the following numbers could your program have found?

→ 108 ms in total → ?? # divisors checked?

21	
24	
29	
33	

Which of the following numbers could your program have found?

→ 108 ms in total → ?? 4 divisors checked?

21	
24	
29	
33	

Which of the following numbers could your program have found?

→ 108 ms in total → ?? 4 divisors checked?

21	2, 3
24	2
29	2, 3, 4, 5
33	2, 3

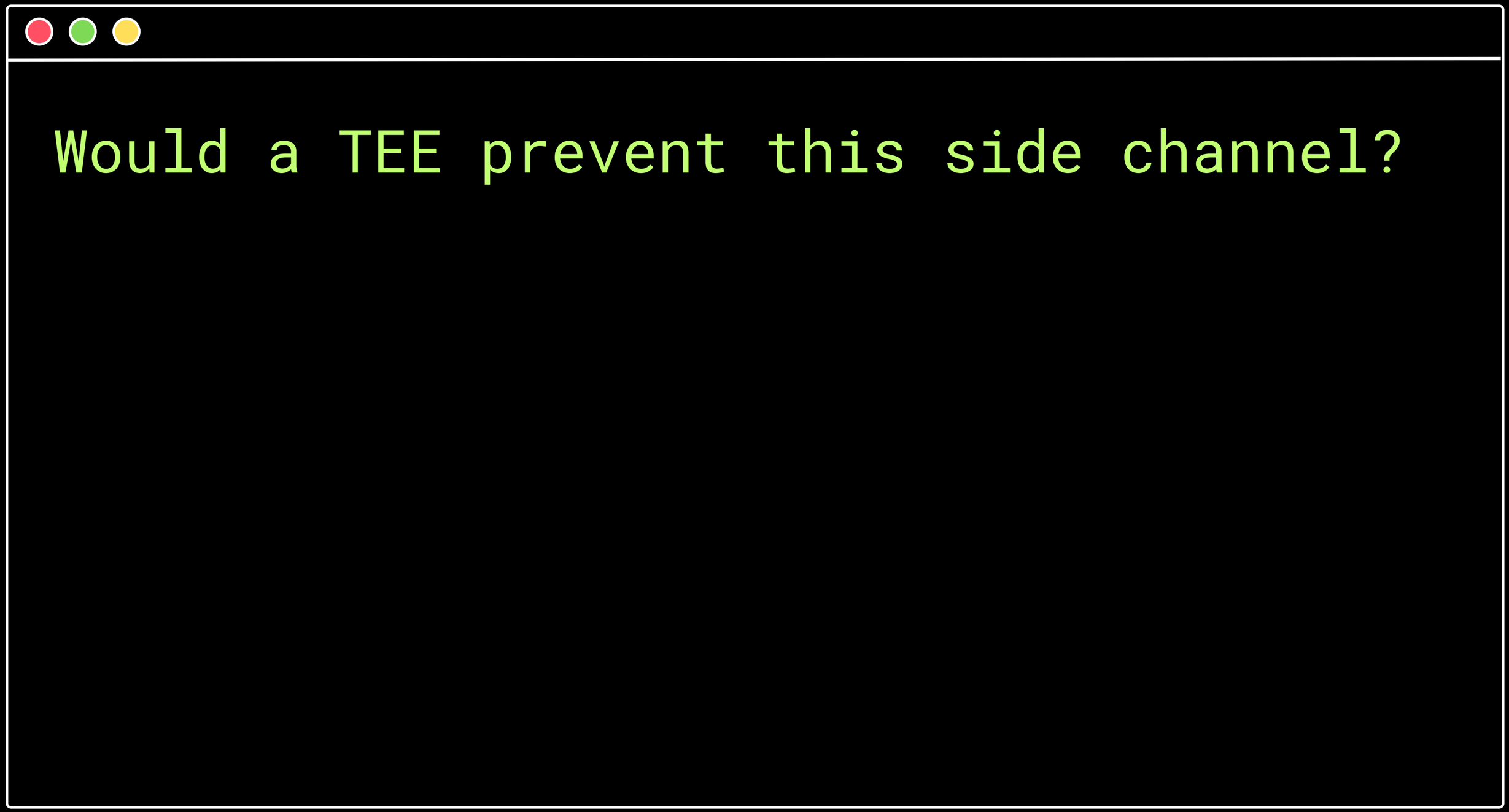
Which … if the total execution time were 104 ms?

21	2, 3
24	2
29	2, 3, 4, 5
33	2, 3

Which … if the total execution time were 104 ms?

→ 21 or 33

21	2, 3
24	2
29	2, 3, 4, 5
33	2, 3


Would a TEE prevent this side channel?

Would a TEE prevent this side channel?

→ No. TEEs protect the confidentiality and integrity of enclave memory but do not necessarily hide execution time.

The Prime Suspect

is a timing side channel

It's Not My Fault!

Q1



Scenario

You're analyzing an application running inside an Intel SGX enclave.

The enclave processes a secret value that determines which of three functions is executed.

The OS cannot directly read the enclave's memory, but it can manipulate page permissions to induce and observe page faults. 👀

```
void common_work() {
    // Perform some computation
}

void option_A() {
    common_work();
}

void option_B() {
    common_work();
}

void option_C() {
    common_work();
}

void process_secret(int secret) {
    if (secret == 0)
        option_A();
    else if (secret == 1)
        option_B();
    else
        option_C();
}
```

Scenario

Assumptions

→ page size is 4 KiB
→ the enclave is loaded with a page-aligned load bias of 0x70000000
→ OS induced a page fault whenever execution enters a different code page
→ consider only page faults caused by these functions

```
void common_work() {
    // Perform some computation
}

void option_A() {
    common_work();
}

void option_B() {
    common_work();
}

void option_C() {
    common_work();
}

void process_secret(int secret) {
    if (secret == 0)
        option_A();
    else if (secret == 1)
        option_B();
    else
        option_C();
}
```

Construct the page-fault trace for each possible value of secret.

→ load bias: 0x70000000
→ page size: 4 KiB
→ We inspect the enclave binary:

Offset	Function
0x1100	process_secret
0x2100	option_A
0x2200	option_B
0x2300	option_C
0x3100	common_work

```
void common_work() {
    // Perform some computation
}

void option_A() {
    common_work();
}

void option_B() {
    common_work();
}

void option_C() {
    common_work();
}

void process_secret(int secret) {
    if (secret == 0)
        option_A();
    else if (secret == 1)
        option_B();
    else
        option_C();
}
```

Construct the page-fault trace for each possible value of secret.

→ Runtime address of a function =
Load bias + Offset

→ Each option calls common_work(), so all three produce identical traces:

0x70001000 → 0x70002000 → 0x70003000 → 0x70002000 → 0x70001000

```
void common_work() {
    // Perform some computation
}

void option_A() {
    common_work();
}

void option_B() {
    common_work();
}

void option_C() {
    common_work();
}

void process_secret(int secret) {
    if (secret == 0)
        option_A();
    else if (secret == 1)
        option_B();
    else
        option_C();
}
```

Can the attacker distinguish what the secret was?

```
void common_work() {
    // Perform some computation
}

void option_A() {
    common_work();
}

void option_B() {
    common_work();
}

void option_C() {
    common_work();
}

void process_secret(int secret) {
    if (secret == 0)
        option_A();
    else if (secret == 1)
        option_B();
    else
        option_C();
}
```

Can the attacker distinguish what the secret was?

→ No. All secret values produce the same trace, so the attacker cannot distinguish them using this information alone.

```
void common_work() {
    // Perform some computation
}

void option_A() {
    common_work();
}

void option_B() {
    common_work();
}

void option_C() {
    common_work();
}

void process_secret(int secret) {
    if (secret == 0)
        option_A();
    else if (secret == 1)
        option_B();
    else
        option_C();
}
```

It's Not My Fault!

Q2



Scenario

The developer slightly modifies the enclave code:

```
void special_work() {
    // Some other computation
}
```

Assume the load bias and all other conditions remain unchanged.

```
void common_work() {
    // Perform some computation
}

void option_A() {
    common_work();
}

void option_B() {
    common_work();
}

void option_C() {
    special_work();
}

void process_secret(int secret) {
    if (secret == 0)
        option_A();
    else if (secret == 1)
        option_B();
    else
        option_C();
}
```

Construct the new page-fault traces for each possible value of secret.

→ load bias: 0x70000000, page size: 4 KiB

→ We inspect the enclave binary:

Offset	Function
0x1100	process_secret
0x2100	option_A
0x2200	option_B
0x2300	option_C
0x3100	common_work
0x4100	special_work

```
void common_work() {
    // Perform some computation
}

void option_A() {
    common_work();
}

void option_B() {
    common_work();
}

void option_C() {
    special_work();
}

void process_secret(int secret) {
    if (secret == 0)
        option_A();
    else if (secret == 1)
        option_B();
    else
        option_C();
}
```

Construct the new page-fault traces for each possible value of secret.

For 0 and 1:
0x70001000 → 0x70002000 → 0x70003000 →
0x70002000 → 0x70001000

Other:
0x70001000 → 0x70002000 → 0x70004000 →
0x70002000 → 0x70001000

```
void common_work() {
    // Perform some computation
}

void option_A() {
    common_work();
}

void option_B() {
    common_work();
}

void option_C() {
    special_work();
}

void process_secret(int secret) {
    if (secret == 0)
        option_A();
    else if (secret == 1)
        option_B();
    else
        option_C();
}
```

Can the attacker distinguish what the secret was?

```
void common_work() {
    // Perform some computation
}

void option_A() {
    common_work();
}

void option_B() {
    common_work();
}

void option_C() {
    special_work();
}

void process_secret(int secret) {
    if (secret == 0)
        option_A();
    else if (secret == 1)
        option_B();
    else
        option_C();
}
```

Can the attacker distinguish what the secret was?

→ The attacker can determine whether the secret belongs to {0, 1} or lies outside this set,but they cannot distinguish 0 from 1.

```
void common_work() {
    // Perform some computation
}

void option_A() {
    common_work();
}

void option_B() {
    common_work();
}

void option_C() {
    special_work();
}

void process_secret(int secret) {
    if (secret == 0)
        option_A();
    else if (secret == 1)
        option_B();
    else
        option_C();
}
```

Does randomizing the load bias prevent you from distinguishing the secret values?

Does randomizing the load bias prevent you from distinguishing the secret values?

→ ASLR changes absolute addresses but preserves relative page offsets. The attacker can therefore distinguish the option functions using their relative positions.

Modify the implementation so that:

→ Its behavior remains unchanged
→ The instruction page-fault trace is independent of secret
→ Unnecessary computation is minimized

```
void common_work() {
    // Perform some computation
}

void option_A() {
    common_work();
}

void option_B() {
    common_work();
}

void option_C() {
    common_work();
}

void process_secret(int secret) {
    if (secret == 0)
        option_A();
    else if (secret == 1)
        option_B();
    else
        option_C();
}
```

Modify the implementation so that:

```
void process_secret(int secret) {
    common_work();
}
```

```
void common_work() {
    // Perform some computation
}

void option_A() {
    common_work();
}

void option_B() {
    common_work();
}

void option_C() {
    common_work();
}

void process_secret(int secret) {
    if (secret == 0)
        option_A();
    else if (secret == 1)
        option_B();
    else
        option_C();
}
```

Other code may execute and additional page faults may occur during trace collection. How might this affect the observed trace, and what challenge does it introduce for the attacker?

Other code may execute and additional page faults may occur during trace collection. How might this affect the observed trace, and what challenge does it introduce for the attacker?

→ This introduces noise, making it harder for the attacker to identify the relevant sequence of page accesses and infer the secret. The attacker may need to filter irrelevant faults or collect multiple traces to distinguish meaningful patterns.

Does this modified implementation prevent the attacker from learning info about secret?

```
uint8_t values[8192];

uint8_t process_secret(uint16_t secret) {
    return values[secret];
}
```

Assumptions

→ Each array element occupies
  1 byte
→ secret is an integer between 0 and 8191
→ Page size is 4 KiB

Does this modified implementation prevent the attacker from learning info about secret?

```
uint8_t values[8192];

uint8_t process_secret(uint16_t secret) {
    return values[secret];
}
```

Assumptions

→ Each array element occupies 1 byte

→ secret is an integer between 0 and 8191

→ Array **values** spans 2 pages (4 KiB = 4096 bytes), so the attacker can determine whether **secret < 4096** or **secret ≥ 4096**

It's Not My Fault!

is a page fault side channel

Q & A